

universität freiburg



CS
DEPARTMENT OF
COMPUTER SCIENCE

Test and Reliability

Chapter 1c: Fault Collapsing

Prof. Dr. Ralf Wimmer
Dr. Tobias Seufert

Summer Term 2026



Goal of this Chapter

- Fault collapsing reduces the number of faults that need to be considered
- Speed up fault simulation and ATPG
- Reduce test patterns

Outline

Introduction

Collapsing equivalent faults

- Fanout-free circuits

- Circuits with fanout

Collapsing dominating faults

- Fanout-free circuits

- Circuits with fanout

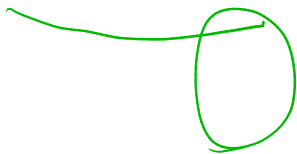
Checkpoint theorem (1976)

Conclusion

Introduction



← coup-complex



Introduction

- Why fault collapsing?
 - Reduce number of faults in order to
 - speed up fault simulation and ATPG
 - shorten test set
- How?
 - Quickly find **representative faults**.
 - Only use these representative faults for fault simulation and ATPG.
- Requirement of fault collapsing algorithm
 - Must be fast; otherwise, it is not worth doing
- There are many different ways of doing fault collapsing
 - Obtaining the **minimum** solution is not so important
 - as long as we do not miss any fault.

Collapsing equivalent faults

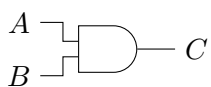


Equivalent faults

Definition (Equivalent faults)

Faults f and g are *functionally equivalent* (or simply *equivalent*), if they cannot be distinguished by any test pattern, i. e., if the faulty circuits behave equivalently.

Example:



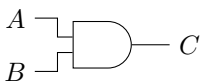
$A/0$, $B/0$, and $C/0$ are all **equivalent faults**.
They belong to the same *equivalent fault class*.

Input		Output						
A	B	good	$A/0$	$B/0$	$C/0$	$A/1$	$B/1$	$C/1$
0	0	0	0	0	0	0	0	1
0	1	0	0	0	0	1	0	1
1	0	0	0	0	0	0	1	1
1	1	1	0	0	0	1	1	1

Properties of Fault Equivalence

- Fault equivalence is **symmetric**:
 - If fault f is equivalent to fault g , then g is equivalent to f .
- Fault equivalence is **transitive**:
 - If fault f is equivalent to fault g and g to fault h , then f is equivalent to h .
- Fault equivalence is **reflexive**:
 - Every fault is equivalent to itself.

Example:



$A/0$ is equivalent to $B/0$, which is equivalent to $C/0$. These three faults belong to the same equivalence class.

Input		Output						
A	B	good	A/0	B/0	C/0	A/1	B/1	C/1
0	0	0	0	0	0	0	0	1
0	1	0	0	0	0	1	0	1
1	0	0	0	0	0	0	1	1
1	1	1	0	0	0	1	1	1

—

—

—

—

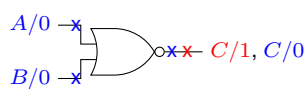
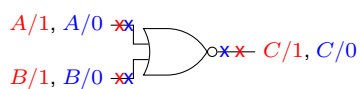
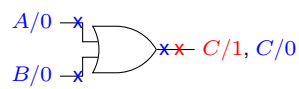
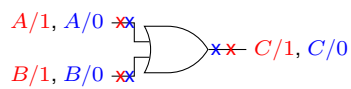
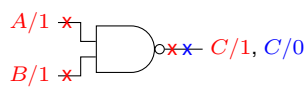
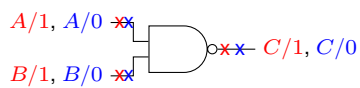
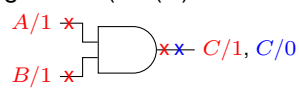
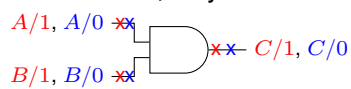
—

—

—

Equivalent fault collapsing (EFC)

- EFC reduces the fault list by using the fault equivalence relationship.
- Select one representative fault from each equivalence class.
- Example:
 - Originally 6 stuck-at faults for each 2-input gate
 - After EFC, only four faults per gate left (for (N)AND and (N)OR):



- Note: EFC is not unique!

 *CDP-complete*

How to perform EFC?

- EFC can be done using **functional analysis** or **structural analysis**
- **Functional analysis:**
 - Prove equivalence of two faulty circuits using verification methods
 - In the worst case: try all possible input patterns and compare outputs
 - If the two outputs always have the same value, faults are equivalent.
 - Typically **too time consuming!**
- **Structural analysis:**
 - Identify equivalent faults according to circuit structure
 - No test patterns applied
 - Typically much faster, but incomplete.

Many different approaches to structural fault analysis available. We will learn a linear-time algorithm, which is not complete, but sufficient for most applications.

Collapsing equivalent faults

Fanout-free circuits



~~AK~~

OR

~~E/0~~

~~F/1~~

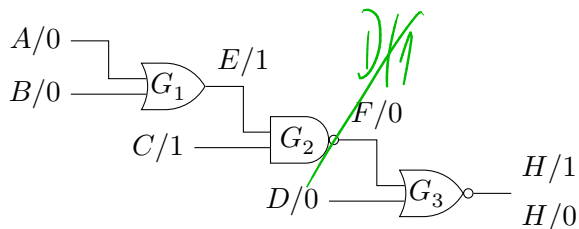
EFC on fanout-free circuits

Current limitation: only fanout-free circuits, i. e., every signal only drives a single gate (tree-shaped circuit).

- Rules:

- both stuck-at-1 and stuck-at-0 faults **every primary output**
- one collapsed fault for each gate input, namely the non-controlling value (1 for (N)AND, 0 for (N)OR).

Example:

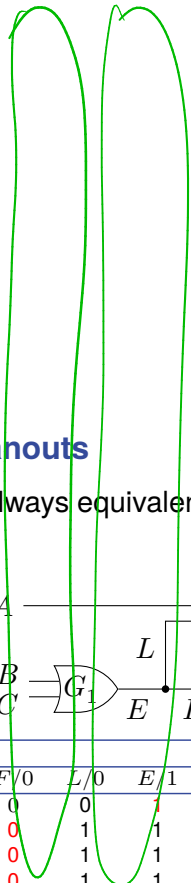
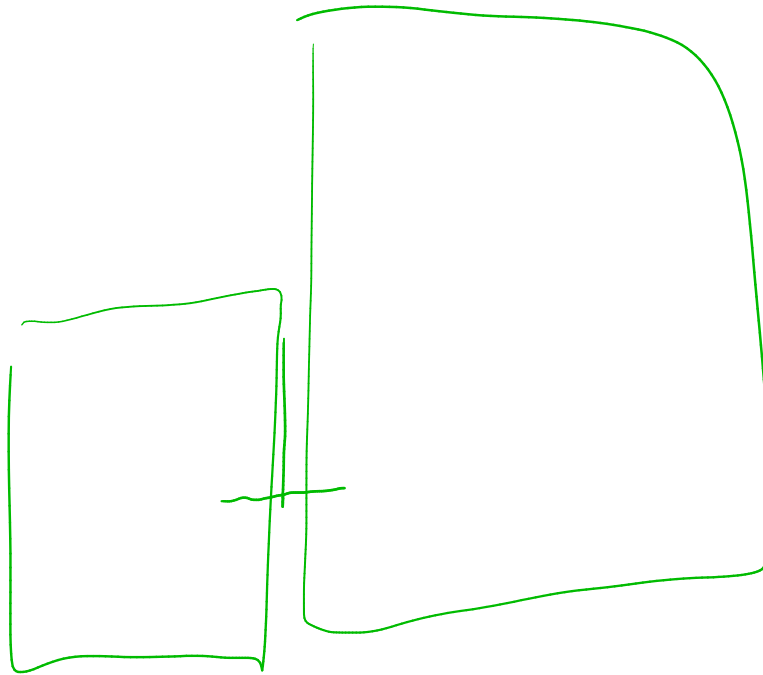


- Keep both $H/0$ and $H/1$ faults for primary output H
- Keep one fault for each gate input
- Original: 14 faults $\rightarrow$ 8 faults after EFC
- Why are faults removed?
 - $A/1 \equiv B/1 \equiv E/1$
 - $E/0 \equiv C/0 \equiv F/1$
 - $F/1 \equiv D/1 \equiv H/0$

Collapsing equivalent faults

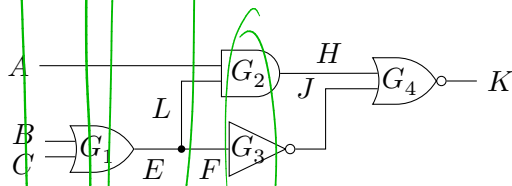
Circuits with fanout





EFC on circuits with fanouts

- Fanout stems are **not** always equivalent to fanout branch faults.
- Example:

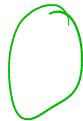


Input			Output						
A	B	C	good	E/0	F/0	L/0	E/1	F/1	L/1
0	0	0	0	0	0	0	1	1	0
0	0	1	1	0	0	1	1	1	1
0	1	0	1	0	0	1	1	1	1
0	1	1	1	0	0	1	1	1	1
1	0	0	0	0	0	0	0	1	0
1	0	1	0	0	0	1	0	0	0
1	1	0	0	0	0	1	0	0	0
1	1	1	0	0	0	1	0	0	0

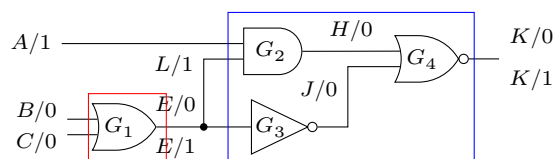
- $E/0$ is equivalent to $F/0$, but not to $L/0$
- Not other faults are equivalent.

How to handle circuits with fanouts?

- **Stem analysis** determines equivalent faults on fanout stem and its branches
 - However, stem analysis is time consuming
 - Typically not worth the time
 - Details skipped in this lecture.
- An approximate solution:
 - Partition the circuit into independent *fanout-free subcircuits*
 - Every fanout stem treated as primary output
 - Both $s@0$ and $s@1$ faults are included on branches.
 - They are **not** collapsed!



Example



- 2 subcircuits
- Originally 18 faults $\rightarrow$ after EFC 10 faults
- Note: Inverter G_3 ignored because input $s@0$ is always equivalent to output $s@1$.

Simple EFC Algorithm

```
1: procedure SIMPLE_EFC( $N$ )
2:   fault_list =  $\emptyset$ 
3:   for all gates, POs, and PIs  $g$  in  $N$  do
4:     if  $g$  is PO or  $g$  is PI and fanout stem then
5:       fault_list = fault_list  $\cup$   $\{g/0, g/1\}$ 
6:     else if output of gate  $g$  is fanout stem then
7:       fault_list = fault_list  $\cup$   $\{g \text{ output}/1, g \text{ output}/0\}$ 
8:     end if
9:     if gate  $g$  is AND or  $g$  is NAND then
10:      fault_list = fault_list  $\cup$   $\{g \text{ input}/1\}$ 
11:    else if gate  $g$  is OR or  $g$  is NOR then
12:      fault_list = fault_list  $\cup$   $\{g \text{ input}/0\}$ 
13:    end if
14:  end for
15: return fault_list
16: end procedure
```

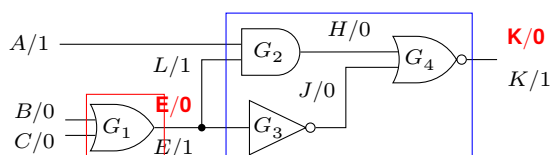
$\triangleright N$ is a netlist

Problems of SIMPLE_EFC (1)

- Does **NOT** guarantee optimal result because it lacks stem analysis.
- In most cases acceptable results.
- Linear running time.

Example

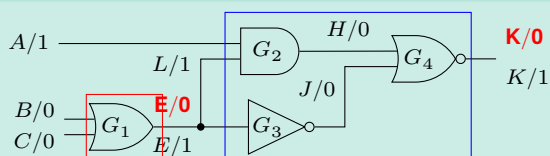
$E/0 \equiv K/0$, but they both appear in the fault list after SIMPLE_EFC.



Problems of SIMPLE_EFC (2)

- Link between original (uncollapsed) faults and their corresponding collapsed fault is lost.
- Example:
 - No link between four faults $\{F/0, J/1, H/1, K/0\}$ in the same equivalence class and their collapsed fault $K/0$.
- Links between uncollapsed faults and collapsed faults are needed when calculating the **uncollapsed fault coverage**.

Example



Suppose we detect $A/1$, $K/0$, and $B/0$

- Collapsed fault coverage: $3/10 = 30\%$
- Uncollapsed fault coverage: $?/18 = ?$

.

Questions

- What about XOR gates?
- How do we modify the SIMPLE_EFC algorithm to keep track of the number of faults in each equivalence class?

Collapsing dominating faults



Fault dominance

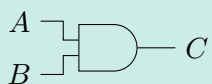
Definition (Detecting set)

The **detecting set** T_f of a fault f is the set of all test patterns that detect fault f .

Definition (Fault dominance)

A fault f **dominates** fault g (written $f \geq g$) if the detecting set of f contains that of g , i. e., $T_f \supseteq T_g$.

Example



$C/1$ dominates $A/1$:

- $T_{C/1} = \{00, 01, 10\}$ contains $T_{A/1} = \{01\}$.
- $C/1$ is the **dominating fault**, $A/1$ the **dominated fault**.

Questions

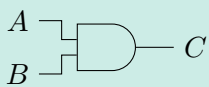
Is fault dominance an equivalence relation, i. e.,

1. Is fault dominance transitive?
2. Is fault dominance symmetric?
3. Is fault dominance reflexive?

Dominance fault collapsing (DFC)

- DFC reduces the fault list using the fault dominance relation.
 - If a test pattern detects a dominated fault, it automatically also detects the dominating fault. So **dominating faults can be removed** for ATPG.

Example

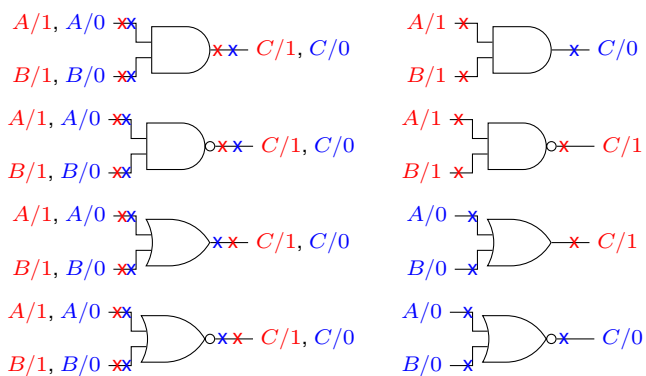


Input		Output						
A	B	good	A/0	B/0	C/0	A/1	B/1	C/1
0	0	0	0	0	0	0	0	1
0	1	0	0	0	0	1	0	1
1	0	0	0	0	0	0	1	1
1	1	1	0	0	0	1	1	1

- Equivalent: $A/0 \equiv B/0 \equiv C/0$
- Dominance: $C/1 \geq A/1$ and $C/1 \geq B/1$.
- Remove $A/0, B/0, C/1$.
- Minimum $\{A/1, B/1, C/0\}$ of three faults remain after DFC and EFC.

DFC on elementary gates

- 2-input (N)AND and (N)OR gates
 - Originally 6 faults $\rightarrow$ 4 faults after EFC $\rightarrow$ 3 faults after DFC
- For n -input elementary gate:
 - $n + 1$ stuck-at faults after EFC and DFC.

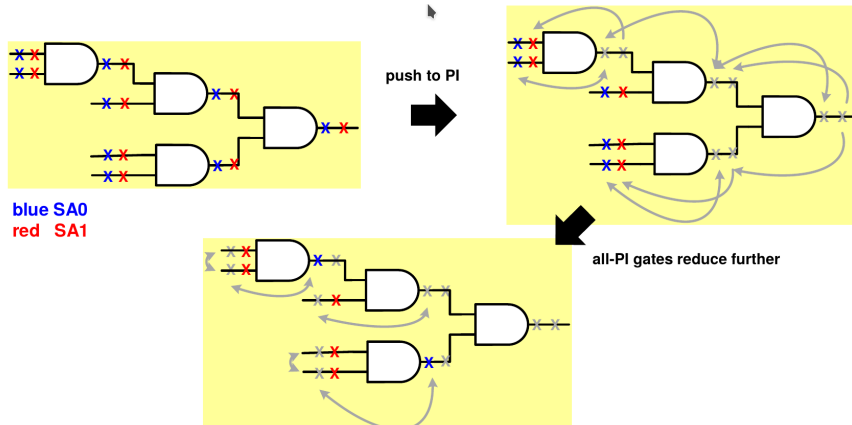


`Collapsing dominating faults`

Fanout-free circuits



DFC on fanout-free circuits



- Two observations:
 - All gate input faults removed — except primary input gates.
 - All gate output faults removed — except all-PI gates.
- This is also applicable to other elementary gates [Why?]

DFC rules

DFC rules:

1. one DFC collapsed fault for every primary input (s-a-non-controlling value)
2. one DFC collapsed fault for every all-PI gate output (s-a-controlling value)

`Collapsing dominating faults`

Circuits with fanout



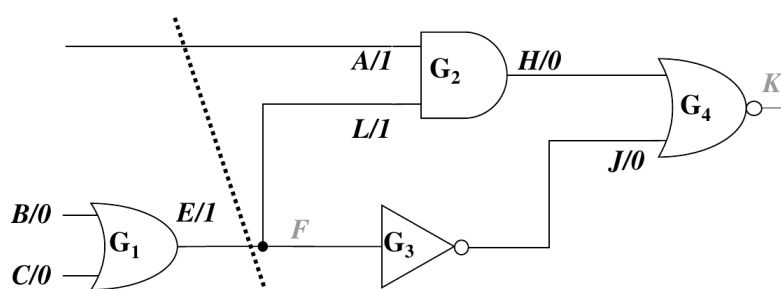
Fanout Stem and Branches

- Fanout branch faults do not always dominate the fanout stem faults.
- Stem analysis of fanout faults is time-consuming.

Approximate solution:

- Partition circuit into fanout-free subregions
- DFC each subregion independently.
- Yields a linear-time algorithm
- Does not guarantee optimality, but in practice acceptable results.

Example: DFC



- J is considered as an input because NOT gate G_3 is ignored.
- Original: 18 faults $\rightarrow$ after EFC 10 faults $\rightarrow$ after DFC 7 faults.
- This solution is NOT optimal:
 $J/0$ is equivalent to $F/1$, which dominates $E/1$.

Checkpoint theorem (1976)

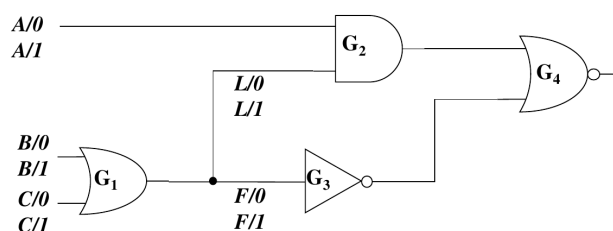


Checkpoint theorem

Checkpoints = primary inputs and fanout branches

Theorem (Breuer '76)

If a test set detects all stuck-at faults on checkpoints, this test set detects all stuck-at faults in the circuit.



- 10 faults on checkpoints
- Originally 18 faults $\rightarrow$ 10 after EFC $\rightarrow$ 7 after DFC
- In general: $|\text{CHKPT}| \geq |\text{EFC}| \geq |\text{DFC}|$

Conclusion

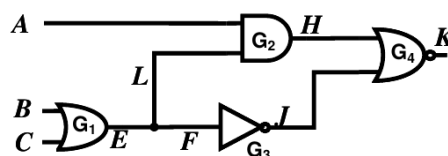


EFC and DFC in practice

Why is DFC typically not used in practice?

- DFC fault coverage is pessimistic. A test pattern can detect a dominating fault, but not a dominated fault.
- Example:

Input			Output						
A	B	C	good	E/0	F/0	L/0	E/1	F/1	L/1
0	0	0	0	0	0	0	1	1	0
0	0	1	1	0	0	1	1	1	1
0	1	0	1	0	0	1	1	1	1
0	1	1	1	0	0	1	1	1	1
1	0	0	0	0	0	0	0	1	0
1	0	1	0	0	0	1	0	0	0
1	1	0	0	0	0	1	0	0	0
1	1	1	0	0	0	1	0	0	0



$ABC = 100$ does not detect dominated fault $E/1$, but it detects dominating fault $F/1$. Since $F/1$ is removed by DFC, it does not count for coverage.

Conclusion

- Fault collapsing reduces the size of the fault list
→ Less time for fault simulation, ATPG etc.
- DFC rules:
 1. one fault on PI (non-controlling value)
 2. one fault on output of all-PI gates (controlling value)
- DFC is more aggressive than EFC, but rarely used in practice, because fault coverage is pessimistic
- Checkpoint theorem:
 - Testing all checkpoint faults is sufficient
 - Checkpoints are (1) PI (2) fanout branches.